Chapter 2
Secure architecture and Principles of secure designing

Abhijeet Pasi

Department of Computer Engineering

K J Somaiya School of Engineering

Outline

- Principles of Secure Design in Software Development
- What does Secure Design in Software Development really mean?
- Code vs. Security – The Security-by-Design Imperative
- 7 Principles of Secure Design in Software Development
- Implement & Ensure Secure Development Practices
- Guidelines for Audit of Applications
- Ensure Secure Application Deployment and Operations
- Summary

Principles of Secure Design in Software Development

- Attackers use advanced techniques that outdated security can't defend against.
- Build security directly into the application design from the start.
- 77% of organizations know their app security is insufficient.
- The struggle is adding security without disrupting established development cycles.
- Secure design is powerful but hard to put into practice.
- Success requires systematically integrating security into the development lifecycle.

What does Secure Design in Software Development really mean?

- "Secure-by-design" builds security into systems from the start.
- It integrates security practices early in the software lifecycle.
- This reduces the attack surface and strengthens products at launch.
- Rooted in DevSecOps, it unifies development, security, and operations.
- It promotes security awareness and shared responsibility across teams.
- While common in software, it also applies to hardware and infrastructure.

Code vs. Security – The Security-by-Design Imperative

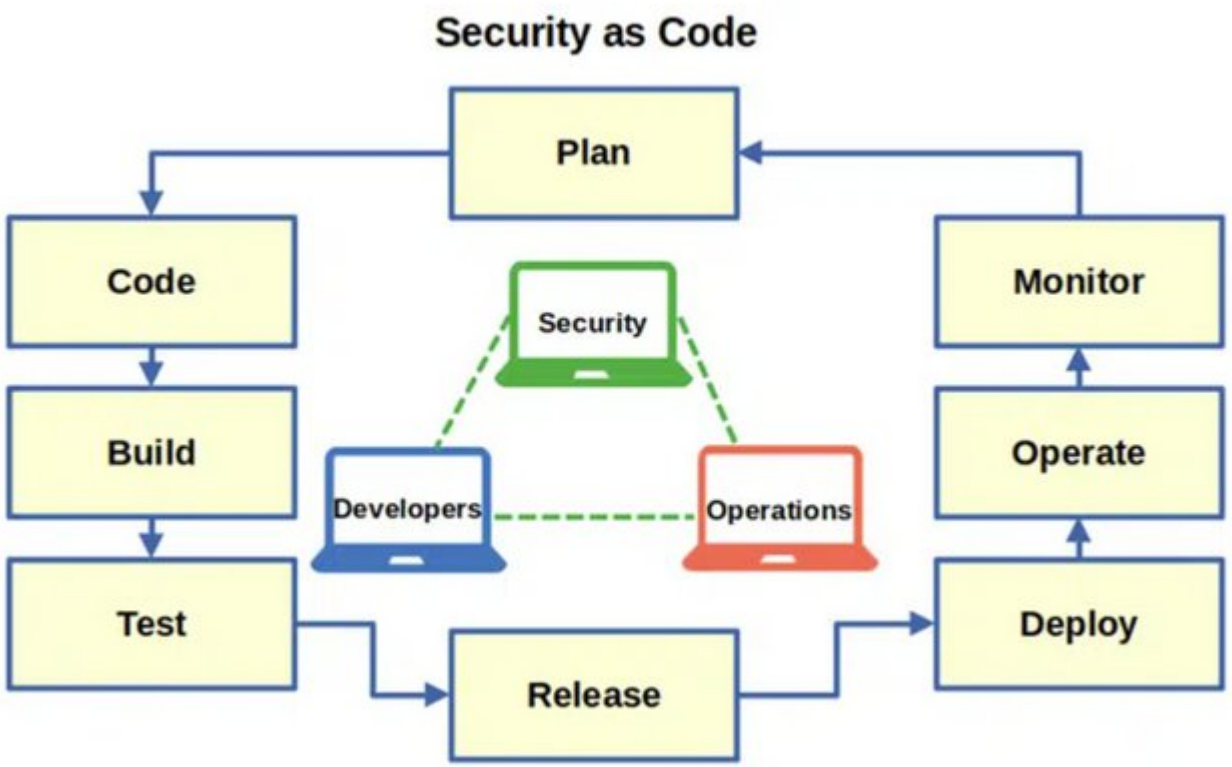
- Fast-release demands often push security to the side, creating risk.
- Secure-by-design embeds protection early, uniting IT and DevOps teams.
- This proactive stance reduces business risk and strengthens security posture.
- It cuts long-term costs and accelerates time to market.
- The approach also improves incident response and code quality.
- It simplifies proving compliance with frameworks like SOC2 and AWS FTR.

7 Principles of Secure Design in Software Development

1. Security as Code
   - Security as Code automates manual security tasks within the development pipeline.
   - It uses scripts and templates to enforce policies and scan for threats.
   - Some platform integrates open-source tools directly into GitHub or AWS.
   - It scans code for vulnerabilities, secrets, dependencies, and misconfigurations.
   - All tools are managed in one interface with real-time results.
   - This approach allows teams to detect and fix vulnerabilities faster.

7 Principles of Secure Design in Software Development

1. Security as



7 Principles of Secure Design in Software Development

2 Secure Defaults

- Secure Defaults mean configuring systems to be safe from the start.
- Most users keep default settings, making secure defaults crucial for protection.
- Examples include strict passwords, MFA requirements, and disabling risky services.
- WordPress once had weak defaults, leaving millions vulnerable to attacks.
- Newer versions now enforce strong defaults like unique database prefixes.
- This shift has significantly reduced user risk and security incidents.

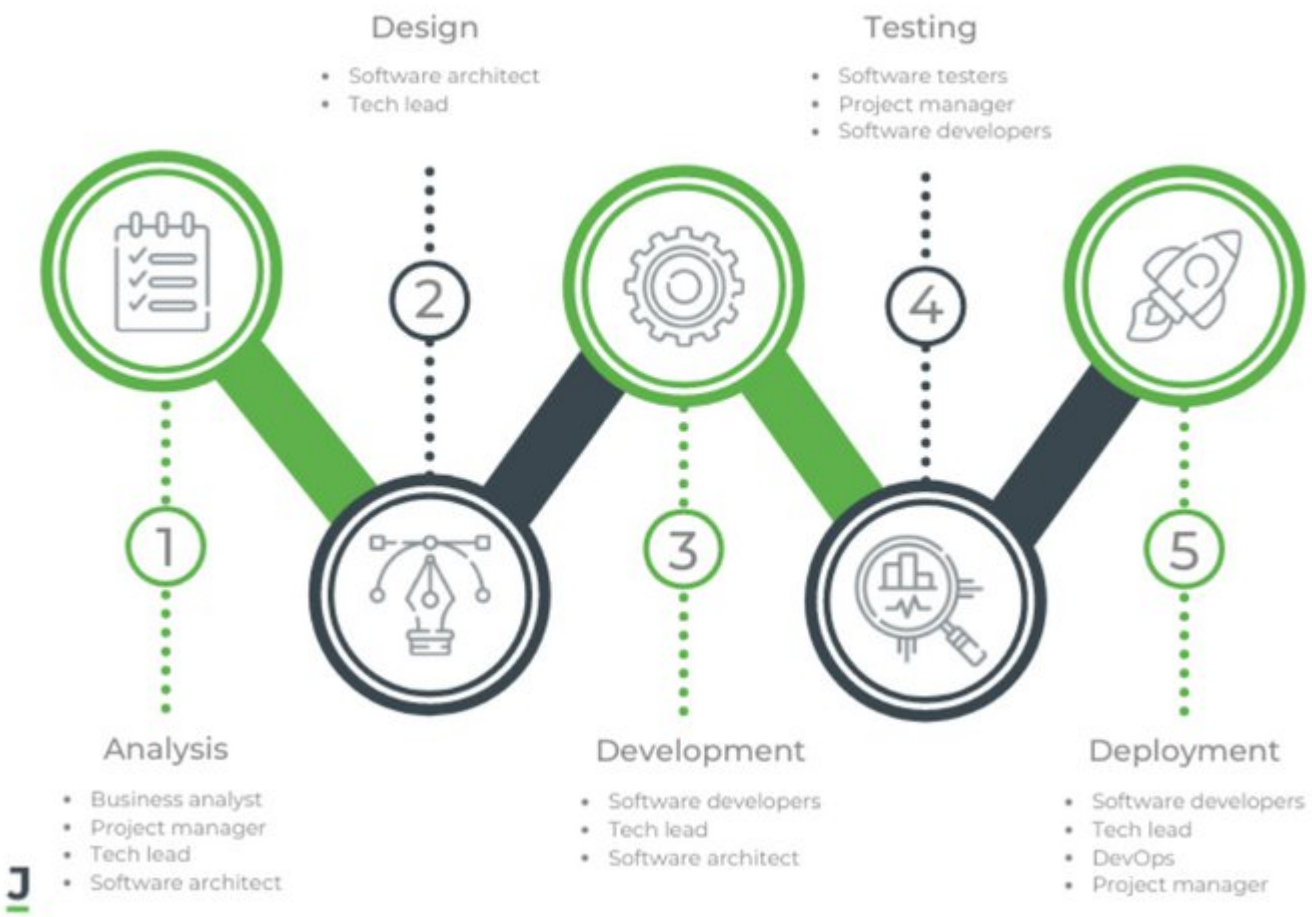
7 Principles of Secure Design in Software Development

3. Least Privilege
   - Least Privilege limits access to the minimum necessary for users and systems.
   - It applies to both human users and machine identities like resources.
   - This practice shrinks the attack surface and limits breach damage.
   - Implement it using precise Role-Based Access Control (RBAC).
   - Regularly review and audit all system and user privileges.
   - Automation tools can scan for and remove unnecessary permissions.

7 Principles of Secure Design in Software Development

4. Separation of Duties
   - Separation of Duties divides tasks and privileges to prevent misuse.
   - It stops any single person from having too much control.
   - In CI/CD, the coder should not also deploy to production.
   - This requires clear access controls and defined workflows.
   - Its goal is to reduce insider risk and accidental errors.
   - Implementation is straightforward but demands strict enforcement.

7 Principles of Secure Design in Software Development



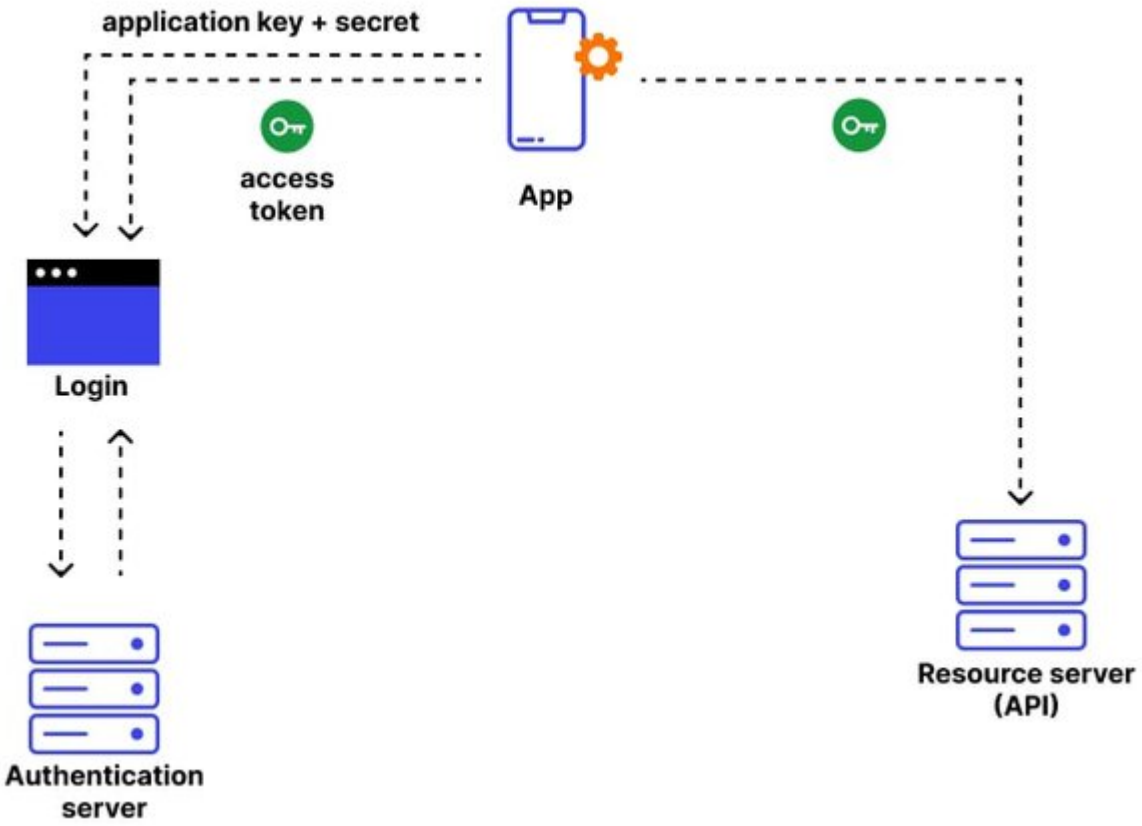
7 Principles of Secure Design in Software Development

5. Minimize Attack Surface Area
   - Attack Surface Area is all points where an attacker can interact.
   - Minimizing ASA reduces the entry points for exploitation.
   - Remove non-essential code, services, and processes from systems.
   - During server setup, only open necessary ports and block the rest.
   - Design APIs carefully—avoid overly broad or wildcard endpoints.
   - Disable and remove unused or deprecated features and components.

7 Principles of Secure Design in Software Development

6. Complete Mediation
   - Complete Mediation requires verifying every access request without exception.
   - Each request must be authenticated and authorized individually.
   - In microservices, every service must check permissions independently.
   - An initial gateway authentication is not sufficient for downstream access.
   - This is often done using a valid authentication token per API call.
   - Each service decodes the token to verify permissions before responding.

7 Principles of Secure Design in Software Development



7 Principles of Secure Design in Software Development

7. Failing Securely
   - Fail Securely means a system must remain safe during errors.
   - Failures should default to a secure state, not expose vulnerabilities.
   - Implement strong error handling to prevent data leaks.
   - Show only generic error messages without internal system details.
   - In authentication, don't reveal if the username or password is wrong.
   - Simply state "authentication failed" to avoid giving attackers hints.

Implement & Ensure Secure Development Practices

- Authentication, Authorization & Session Management
- Cryptographic Practices
- Version Control and Change Management
- Ensure Secure Coding
- File & Memory Management
- Optimise to Manage Complexity
- Develop Software Technology Specific Security Checklist
- Security Test Driven Development (STDD)
- Implementation of Threat Modelling in Application Development

Implement & Ensure Secure Development Practices

- Build a Secure Environment for Application Development:
- Secure Use of Environment Variables
- Stored Procedures Over SQL Statements:
- Handle Error Messages, Commented Code, and Exceptions
- Linear Data Structure and Multiple Inheritances
- Third Party and Open-Source Libraries, Components, and APIs
- Build Trust Boundaries
- Principle of Least Privileges
- Enhancing Maturity of Software Security

Guidelines for Audit of Applications

- Source Code Review
- Conduct Security Vulnerability Assessment
- Timeline for Completion of Audit
- Penetration Testing:
- Logging and Audit Trails:
- Precondition for Assessment and Audit:

Ensure Secure Application Deployment and Operations

- Secure Deployment and Configuration:
- Provision for Patch and Update
- Secure Development of Update, Patch & Release to Mitigate Against Supply Chain Risk from Developers

Summary

- Secure-by-Design: Build security into the system from the start of development.
- Security as Code: Automate security checks and policies directly into the development pipeline.
- Secure Defaults: Configure default settings to be highly secure right out of the box.
- Least Privilege: Limit access for users and systems to only what is absolutely necessary.
- Separation of Duties: Divide tasks to prevent any single person from having too much control.
- Minimize Attack Surface: Reduce entry points by eliminating unused code, services, and open ports.
- Complete Mediation: Authenticate and authorize every request—no exceptions.
- Fail Securely: Keep systems safe even during errors; avoid exposing sensitive information.
- Benefits: Faster releases, lower risk, reduced costs, easier compliance, and stronger overall security posture.